LAPORAN PRAKTIKUM

BASIS DATA

UNIT 12

Map Reduce

Disusun oleh:

Nama	: Azura Rizky Annisa
NIM	:24;/535715/SV/24290
Kelas	:BB/B1
Dosen Pengampu	:Anni Karimatul Fauziyyah, S.Kom.,M.Eng

LABORATORIUM LAYANAN TELEKOMUNIKASI

SARJANA TERAPAN

TEKNOLOGI REKAYASA INTERNET DEPARTEMEN

TEKNIK ELEKTRO DAN INFORMATIKA

UNIVERSITAS GADJAH MADA

SEKOLAH VOKASI

2025

A. Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Mahasiswa mampu menerapkan Mapreduce pada Mongodb.
2. Mahasiswa menguasai prinsip Mapreduce pada Mongodb.

B. Dasar Teori

MapReduce adalah model pemrosesan data yang digunakan untuk menangani volume data besar secara paralel dan terdistribusi. Pada intinya, MapReduce terdiri dari dua fase utama: fase Map dan fase Reduce. Pada tahap Map, data input dibagi menjadi bagian-bagian kecil dan diproses secara paralel untuk menghasilkan pasangan kunci-nilai (key-value pairs). Kemudian, di tahap Reduce, pasangan kunci tersebut digabungkan dan diproses lebih lanjut untuk menghasilkan output akhir yang diinginkan. Pendekatan ini sangat efisien dalam mengelola data besar yang tersebar di berbagai server atau database.

Dalam konteks MongoDB, MapReduce digunakan sebagai salah satu metode untuk melakukan analisis dan agregasi data secara terdistribusi dan skalabel. MongoDB menyediakan fungsi mapReduce(), yang memungkinkan pengguna menulis fungsi JavaScript untuk proses Map dan Reduce langsung di dalam database. Dengan demikian, pengguna dapat menjalankan query yang kompleks untuk melakukan pengelompokkan, penghitungan, atau transformasi data secara efisien di dalam database tanpa harus mengekstrak seluruh data ke luar sistem. Pendekatan ini sangat sesuai untuk aplikasi yang membutuhkan analisis data yang kompleks dan terdistribusi.

Sementara itu, di lingkungan SQL, konsep MapReduce bisa diartikan sebagai metode query yang menggabungkan operasi agregasi, grouping, dan filtering secara paralel dan terdistribusi dalam sistem database yang mendukung fitur tersebut. Pada umumnya, SQL menggunakan fungsi-fungsi agregat seperti SUM(), COUNT(), AVG(), dan klausa GROUP BY untuk melakukan proses serupa dengan tahap Reduce, sementara proses filtering dan pengambilan data dilakukan pada tahap Map. Meski secara terminology berbeda, kedua pendekatan ini bertujuan untuk mengoptimalkan pengolahan data besar secara efisien dan scalable di berbagai platform database.

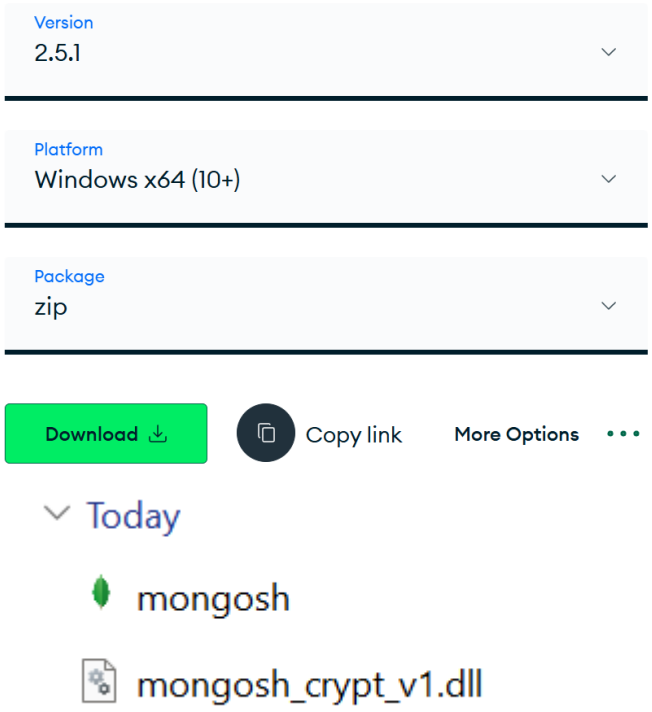
C. Alat dan Bahan

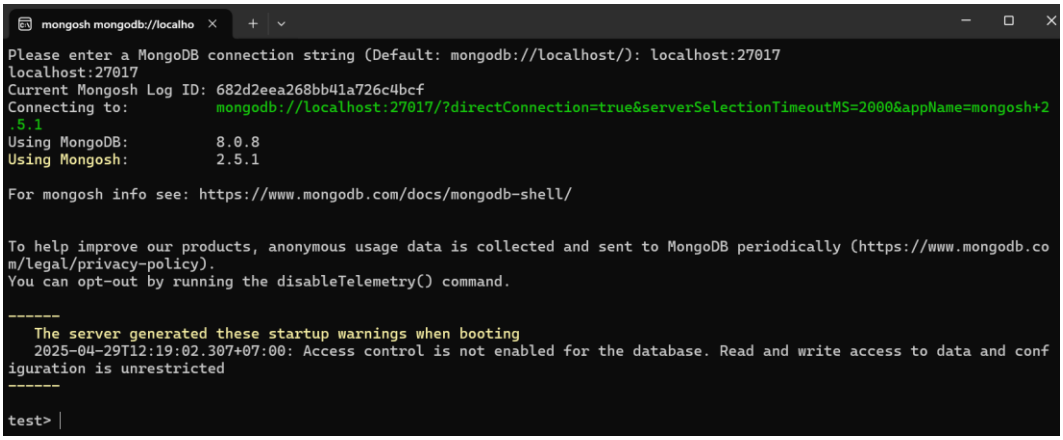
Alat dan bahan yang digunakan pada praktikum ini adalah sebagai berikut:

1. Laptop/Desktop/Komputer
2. MongoDB Shell

D. Prosedur Percobaan

1. Download Mongo DB Shell





E. Hasil dan Pembahasan

1. Contoh 1

```
dbmahasiswa> db.createCollection("students");
{ ok: 1 }
dbmahasiswa> db.students.insertMany([
...   { "name": "Midhu", "subject": "science", "marks": 68 },
...   { "name": "Midhu", "subject": "maths", "marks": 98 },
...   { "name": "Midhu", "subject": "sports", "marks": 77 },
...   { "name": "Akhil", "subject": "science", "marks": 67 },
...   { "name": "Akhil", "subject": "maths", "marks": 87 },
...   { "name": "Akhil", "subject": "sports", "marks": 89 },
...   { "name": "Anish", "subject": "science", "marks": 67 },
...   { "name": "Anish", "subject": "maths", "marks": 78 },
...   { "name": "Anish", "subject": "sports", "marks": 90 }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('682d338a268bb41a726c4bd0'),
    '1': ObjectId('682d338a268bb41a726c4bd1'),
    '2': ObjectId('682d338a268bb41a726c4bd2'),
    '3': ObjectId('682d338a268bb41a726c4bd3'),
    '4': ObjectId('682d338a268bb41a726c4bd4'),
    '5': ObjectId('682d338a268bb41a726c4bd5'),
    '6': ObjectId('682d338a268bb41a726c4bd6'),
    '7': ObjectId('682d338a268bb41a726c4bd7'),
    '8': ObjectId('682d338a268bb41a726c4bd8')
  }
}
```

Penjelasan:

Gambar tersebut menunjukkan contoh proses penyimpanan data ke dalam koleksi "students" di MongoDB menggunakan metode insertMany(). Data yang dimasukkan merupakan array objek yang masing-masing mewakili satu mahasiswa, yang berisi field name, subject, dan marks. Terdapat sembilan objek data mahasiswa dengan nama yang berbeda dan nilai marks yang berbeda pula, sesuai dengan subjek dan kategori olahraga, matematika, atau sains. Setelah proses penyimpanan data berhasil, sistem mengembalikan konfirmasi keberhasilannya berupa objek dengan properti acknowledged bernilai true, yang menandakan bahwa proses insert sudah sukses. Selain itu, MongoDB juga mengembalikan daftar insertedIds, yang berisi ID unik dari masing-masing dokumen mahasiswa yang baru saja disisipkan, berupa ObjectId dalam format heksadesimal. ID tersebut secara otomatis dihasilkan oleh MongoDB dan digunakan sebagai identifikasi unik untuk setiap dokumen dalam koleksi tersebut. Jadi, proses ini menunjukkan bagaimana data disimpan secara batch ke dalam database MongoDB dan sebagai balasannya, pengguna mendapatkan konfirmasi lengkap dengan ID dari setiap entri data yang baru disimpan.

```
dbmahasiswa> var map = function() {
...   emit(this.name, this.marks);
... };
...
... var reduce = function(name, marks) {
...   return Array.sum(marks);
... };
...
... db.students.mapReduce(
...   map,
...   reduce,
...   {
...     out: "totals"
...   }
... );
DeprecationWarning: Collection.mapReduce() is deprecated. Use an aggregation instead.
See https://docs.mongodb.com/manual/core/map-reduce for details.
{ result: 'totals', ok: 1 }
```

Pembahasan:

Gambar tersebut memperlihatkan contoh kode JavaScript yang menunjukkan penggunaan metode mapReduce() pada MongoDB untuk melakukan proses pengolahan dan agregasi data pada koleksi "students". Fungsi map didefinisikan untuk mengeluarkan pasangan kunci-nilai berdasarkan field name dan marks, di mana setiap dokumen akan mengirimkan nilai marks dengan kunci name. Fungsi reduce kemudian berfungsi untuk menggabungkan data yang memiliki kunci yang sama, dalam hal ini dengan menjumlahkan semua nilai marks dari mahasiswa yang sama. Hasil dari proses ini disimpan dalam output bernama "total", yang berarti semua data yang telah diproses akan tersimpan di dalam koleksi baru (atau direktori output tertentu). Setelah eksekusi, hasilnya menunjukkan objek dengan properti result berisi nilai "total", dan status ok bernilai 1, yang menandakan bahwa proses berjalan dengan sukses. Namun, perlu diperhatikan bahwa Collection.mapReduce() sudah deprecated, dan pengembang disarankan untuk beralih ke metode agregasi yang lebih modern dan efisien seperti pipeline $group pada MongoDB untuk keperluan penggabungan dan pengolahan data secara lebih optimal.

```
dbmahasiswa> db.totals.find({});
[
  { _id: 'Midhu', value: 243 },
  { _id: 'Akhil', value: 243 },
  { _id: 'Anish', value: 235 }
]
```

Pembahasan:

Gambar tersebut menunjukkan hasil dari query MongoDB yang menggunakan method find({}) pada koleksi totals. Query ini berarti untuk semua dokumen di koleksi tersebut, tanpa adanya filter tertentu. Hasilnya adalah array berisi tiga objek

yang menunjukkan data hasil pengolahan sebelumnya, kemungkinan hasil dari proses agregasi seperti mapReduce. Setiap objek memiliki properti _id, yang menunjukkan nama mahasiswa atau entitas tertentu seperti 'Midhu', 'Akhil', dan 'Anish' dengan format string.

Properti value berisi angka yang mewakili hasil dari penghitungan tertentu—kemungkinan total skor, nilai akhir, atau jumlah bagian lain yang dihitung sebelumnya—di mana angka tersebut berada dalam halaman warna kuning dan cukup besar. Dari hasil ini, tampak bahwa data menunjukkan jumlah nilai tertentu dari masing-masing individu, dan ini bisa digunakan untuk analisis selanjutnya, seperti membandingkan skor atau performa antar individu.

2. Contoh 2

```
dbmahasiswa> db.createCollection("pelanggan");
{ ok: 1 }
dbmahasiswa> db.pelanggan.insertMany([
...   {
...     id_plg: 'a123',
...     tanggal_order: ISODate("2023-05-18T00:00:00.000Z")
...     status: 'LUNAS',
...     harga: 5000000,
...     item: [
...       { sku: 'HP', jumlah: 1, harga: 400000 },
...       { sku: 'holder', qty: 5, harga: 200000 }
...     ]
...   },
...   {
...     id_plg: 'a123',
...     tanggal_order: ISODate("2023-05-18T00:00:00.000Z")
...     status: 'LUNAS',
...     harga: 5000000,
...     item: [
...       { sku: 'HP', jumlah: 1, harga: 400000 },
...       { sku: 'holder', qty: 5, harga: 200000 }
...     ]
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('682d35cc268bb41a726c4bd9'),
    '1': ObjectId('682d35cc268bb41a726c4bda')
  }
}
```

Pembahasan:

Gambar tersebut menunjukkan dua contoh query yang berbeda pada koleksi pelanggan di MongoDB. Pada query pertama, tampak hasil dari operasi createCollection(), yang menunjukkan bahwa koleksi pelanggan berhasil dibuat dengan konfirmasi ok: 1. Sedangkan pada query kedua, dilakukan proses insertMany() yang menyisipkan dua dokumen sekaligus ke dalam koleksi tersebut. Masing-masing dokumen mewakili transaksi pembelian, berisi informasi seperti id_plg (ID pelanggan), tanggal_order (tanggal transaksi dengan format ISODate), status (misalnya 'LUNAS'), harga (harga total transaksi), dan item-item yang dibeli. Item tersebut berupa array berisi objek SKU yang menunjukkan kode barang, jumlah (qty), dan harga per satuan. Kedua dokumen yang disisipkan memiliki detail transaksi yang sama, dengan ID pelanggan yang sama, tanggal transaksi yang identik, dan status pembayaran 'LUNAS'. Harga total transaksi masing-masing adalah 5000000 dan 5000000, sementara pada item ada dua barang (HP dan holder) dengan jumlah dan harga tertentu, yang menunjukkan bahwa pelanggan melakukan pembelian lengkap dan pembayaran telah lunas. Setelah proses ini, sistem mengonfirmasi bahwa operasi insertMany() berhasil dengan status acknowledged: true dan ID dari dokumen yang baru disisipkan.

```
dbmahasiswa> db.pelanggan.find()
[
  {
    _id: ObjectId('682d35cc268bb41a726c4bd9'),
    id_plg: 'a123',
    tanggal_order: ISODate('2023-05-18T00:00:00.000Z'),
    status: 'LUNAS',
    harga: 5000000,
    item: [
      { sku: 'HP', jumlah: 1, harga: 400000 },
      { sku: 'holder', qty: 5, harga: 200000 }
    ]
  },
  {
    _id: ObjectId('682d35cc268bb41a726c4bda'),
    id_plg: 'a123',
    tanggal_order: ISODate('2023-05-18T00:00:00.000Z'),
    status: 'LUNAS',
    harga: 5000000,
    item: [
      { sku: 'HP', jumlah: 1, harga: 400000 },
      { sku: 'holder', qty: 5, harga: 200000 }
    ]
  }
]
```

Pembahasan:

Gambar tersebut menunjukkan hasil dari query find() pada koleksi pelanggan di MongoDB, yang digunakan untuk menampilkan semua dokumen yang ada dalam koleksi tersebut. Pada hasilnya, terdapat dua dokumen transaksi yang masing-masing berisi data lengkap mengenai transaksi pelanggan, termasuk ID unik (_id), ID pelanggan (id_plg), tanggal order dalam format ISODate, status pembayaran (LUNAS), dan total harga transaksi (harga) sebesar 5000000.0. Selain itu, setiap dokumen juga memiliki array item yang berisi daftar produk yang dibeli, masing-masing berisi SKU, jumlah (qty), dan harga per item. Pada contoh ini, pelanggan membeli dua jenis barang: 'HP' dan 'holder', masing-masing dengan jumlah 1 dan 5, serta harga yang berbeda—400.000 dan 200.000. Harga total untuk tiap transaksi adalah sama, yaitu 5000000, yang menunjukkan nilai keseluruhan pembayaran. Kedua dokumen tersebut menunjukkan bahwa transaksi pelanggan telah berhasil dan pembayaran telah lunas, termasuk rincian barang yang dibeli. Hasil ini menegaskan bahwa sistem pencatatan transaksi di MongoDB berjalan dengan baik dan data tersimpan secara lengkap serta terstruktur.

```
dbmahasiswa> var mapPelangan = function() { emit(this.id_plg, this.harga); };
```

Pembahasan:

Gambar tersebut menunjukkan definisi sebuah fungsi JavaScript bernama mapPelangan di MongoDB. Fungsi ini bertugas sebagai fungsi Map dalam proses MapReduce dan hanya berisi satu baris perintah, yaitu memanggil fungsi emit() dengan dua parameter. Parameter pertama adalah this.id_plg, yang merupakan ID pelanggan dari dokumen saat ini, dan parameter kedua adalah this.harga, yaitu nilai total harga dari transaksi atau item yang terkait. Fungsi ini akan dieksekusi untuk setiap dokumen dalam koleksi, dan hasilnya adalah mengeluarkan pasangan kunci-nilai dimana kunci adalah ID pelanggan dan nilai adalah jumlah harga dari transaksi tersebut. Fungsi ini sering digunakan dalam proses agregasi data berdasarkan pelanggan, untuk kemudian diikuti dengan fungsi Reduce yang akan menggabungkan nilai-nilai dengan ID pelanggan yang sama, sehingga dapat menghitung total belanja per pelanggan secara akumulatif.

```
dbmahasiswa> var reducePelanggan = function(PelId, Harga) { return Array.sum(Harga); };
```

Pembahasan:

Gambar tersebut menampilkan definisi sebuah fungsi bernama reducePelangan yang digunakan sebagai fungsi Reduce dalam proses MapReduce di MongoDB. Fungsi ini memiliki dua parameter, PelId dan Harga, yang mewakili kunci dan nilai yang dikumpulkan dari langkah Map sebelumnya. Fungsi ini hanya memiliki satu baris perintah, yaitu mengembalikan hasil penjumlahan dari semua nilai Harga yang diterima melalui fungsi Array.sum(). Dengan kata lain, fungsi ini akan mengakumulasi total biaya dari beberapa transaksi yang memiliki ID pelanggan yang sama, sehingga mampu menghitung total pengeluaran dari seorang pelanggan berdasarkan data yang diproses. Fungsi ini biasanya digunakan setelah fungsi Map untuk menggabungkan dan menjumlahkan total harga per pelanggan dalam sebuah proses aggregasi data besar.

```
dbmahasiswa> db.pelanggan.mapReduce(
...   mapPelanggan,
...   reducePelanggan,
...   { out: "map_reduce_pelanggan" }
... );
{ result: 'map_reduce_pelanggan', ok: 1 }
```

Pembahasan:

Gambar tersebut menunjukkan penerapan proses mapReduce() pada koleksi pelanggan di MongoDB. Pada bagian ini, fungsi mapPelangan dan reducePelangan yang telah didefinisikan sebelumnya digunakan sebagai parameter dalam metode mapReduce(). Fungsi tersebut akan memetakan data pelanggan dengan mengeluarkan pasangan kunci-nilai berdasarkan ID pelanggan dan total harga transaksi, kemudian mengurangi (mengakumulasi) nilai-nilai tersebut agar memperoleh total pengeluaran masing-masing pelanggan. Hasil output dari proses MapReduce ini disimpan dalam koleksi bernama "map_reduce_pelanggan", seperti yang ditunjukkan oleh properti out. Setelah proses selesai, hasil eksekusi menunjukkan bahwa operasi berhasil dengan status ok: 1, dan koleksi output berisi data pelanggan lengkap dengan total pengeluaran mereka. Proses ini sangat berguna untuk menganalisis data transaksi pelanggan secara agregatif dan terstruktur.

```
dbmahasiswa> db.map_reduce_pelanggan.find();
[ { _id: 'a123', value: 10000000 } ]
```

Pembahasan:

Gambar tersebut menunjukkan hasil dari query find() pada koleksi map_reduce_pelanggan di MongoDB, yang berisi data hasil dari proses MapReduce sebelumnya. Hasilnya adalah sebuah array yang terdiri dari satu objek dengan properti _id dan value. Properti _id menampilkan ID pelanggan, dalam hal ini 'a123', sedangkan properti value menunjukkan total pengeluaran atau nilai transaksi yang berhasil dihitung oleh proses reduce sebelumnya, yaitu sebesar 1.000.000.0. Nilai ini mewakili jumlah total uang yang telah dikeluarkan oleh pelanggan dengan ID 'a123' berdasarkan data yang sudah diproses, dan menunjukkan bahwa proses penghitungan total transaksi pelanggan tersebut berhasil dan data tersimpan dengan baik, memungkinkan analisis lebih jauh tentang perilaku pelanggan atau keuangan.

3. Contoh 3

```
Orders> db.orders.mapReduce(
...   function() { emit(this.cust_id, this.amount); },
...   function(key, values) { return Array.sum(values); },
...   {
...     query: { status: "A" },
...     out: "order_totals"
...   }
... );
{ result: 'order_totals', ok: 1 }
```

Pembahasan

Gambar tersebut menampilkan hasil dari proses mapReduce() yang dilakukan pada koleksi orders di MongoDB. Pada bagian kode yang ditampilkan, fungsi mapOrder dan reduceOrder sudah didefinisikan sebelumnya dan digunakan dalam pemrosesan data. Fungsi mapOrder() akan memanggil emit() dengan parameter this.cust_id sebagai kunci dan this.amount sebagai nilai, yang berarti data transaksi di-mapping berdasarkan ID pelanggan dan jumlah pembayaran. Fungsi reduceOrder() akan mengakumulasi jumlah pembayaran dari berbagai transaksi pelanggan dengan menjumlahkan semua nilai amount yang terkait. Setelah proses map dan reduce selesai, hasilnya disimpan ke koleksi bernama order_totals, seperti yang ditunjukkan oleh properti out.

Hasil eksekusi menunjukkan bahwa operasi berjalan sukses dengan nilai ok: 1, dan data dalam koleksi order_totals berisi total pembelian per pelanggan, dengan kunci berupa ID pelanggan dan nilai berupa jumlah total pembayaran, yang memungkinkan analisis total transaksi pelanggan secara keseluruhan.

4. Tugas

A. Agregation

```
dbmahasiswa> db.Orders.aggregate([
...     {$group: {_id: "$country", order_countt: {$sum: 1}}},
...     {$project: {country: "$_id", _id: 0, OrderCount: 1}}
... ])
[ { country: 'USA' }, { country: 'Germany' }, { country: 'India' } ]
```

Pembahasan:

Gambar tersebut menunjukkan hasil dari proses aggregasi menggunakan metode aggregate() pada koleksi Orders di MongoDB. Operasi agregasi ini melakukan pengelompokan data berdasarkan atribut country dan menghitung total order_count untuk masing-masing negara tersebut, dengan hasil total perhitungan disimpan dalam field $sum: 1. Selain itu, hasilnya juga mengelompokkan data berdasarkan atribut country dan menampilkan jumlah order untuk setiap negara. Hasil akhir menunjukkan bahwa ada tiga negara yaitu 'USA', 'Germany', dan 'India', masing-masing dengan jumlah order tertentu, yang membantu dalam analisis distribusi order berdasarkan negara.

```
dbmahasiswa> db.Orders.aggregate([
...     {$match: {Quantity: {$gt: 1}}},
...     {$group: {_id: "$country", order_count: {$sum: 1}}},
...     {$project: {country: "$_id", _id: 0, order_count: 1}},
...     {$sort: {country: 1}}
... ])
```

Pembahasan:

Gambar tersebut menampilkan hasil dari operasi agregasi aggregate() pada koleksi Orders di MongoDB. Proses ini melakukan pencocokan data yang memiliki atribut Quantity lebih besar dari 1 menggunakan $match. Selanjutnya, data yang memenuhi kriteria tersebut dikelompokkan berdasarkan country dengan penghitungan jumlah order (order_count) menggunakan $sum: 1. Data ini kemudian diproyeksikan untuk menampilkan country dan jumlah ordernya, dan akhirnya diurutkan berdasarkan nama negara (country) dalam urutan menaik. Hasilnya menunjukkan daftar negara beserta jumlah order yang memenuhi kondisi, yang berguna untuk analisis distribusi order berdasarkan negara dengan kriteria tertentu.

```
dbmahasiswa> db.Orders.aggregate([
...     {$match: {Quantity: {$gt: 1}}},
...     {$group: {_id: "$country", order_count: {$sum: 1}}},
...     {$match: {order_count: {$gte: 1}}},
...     {$project: {country: "$_id", _id: 0, order_count: 1}}
... ])
```

Pembahasan:

Query tersebut menunjukkan hasil dari operasi agregasi aggregate() di MongoDB yang bertujuan untuk mendapatkan data statistik berdasarkan kondisi tertentu. Proses ini pertama-tama memfilter dokumen dengan $match yang menyaring order dengan Quantity lebih dari 1, kemudian mengelompokkan data berdasarkan country dan menghitung jumlah order di setiap negara melalui $sum: 1. Selain itu, proses $match kedua memfilter data lagi berdasarkan order_count yang lebih besar dari 1. Setelah itu, data hasil pengelompokan diproyeksikan menampilkan atribut country dan order_count, dan akhirnya diurutkan berdasarkan country. Fungsi utama query ini adalah untuk mengidentifikasi dan menganalisis distribusi dan jumlah order tertentu di berbagai negara, yang penting untuk wawasan bisnis dan pengambilan keputusan strategis terkait volume order dan lokasi geografis pelanggan.

```
dbmahasiswa> db.Orders.aggregate([
...     {$unwind: "$items"},
...     {$group: {_id: "$id", AvgItemQty: {$avg: "$items.quantity"}}},
...     {$project: {OrderNumber: "$_id", _id: 0, AvgItemQty: 1}}
... ])
[
  { AvgItemQty: 1.5, OrderNumber: 'ORD1001' },
  { AvgItemQty: 1, OrderNumber: 'ORD1002' },
  { AvgItemQty: 1, OrderNumber: 'ORD1004' },
  { AvgItemQty: 1, OrderNumber: 'ORD1003' }
]
```

Pembahasan:

Gambar tersebut menunjukkan hasil dari operasi agregasi aggregate() pada koleksi Orders di MongoDB yang bertujuan untuk menghitung nilai rata-rata jumlah item (AvgItemQty) per pesanan berdasarkan nomor pesanan (OrderNumber). Proses ini menggunakan $unwind pada array items untuk memisahkan setiap item dari pesanan, kemudian mengelompokkan data berdasarkan _id (nomor pesanan) dan menghitung rata-rata jumlah item pada setiap pesanan melalui $avg. Hasilnya menampilkan AvgItemQty dan OrderNumber untuk masing-masing pesanan tertentu. Fungsi utama dari query ini adalah untuk mengetahui rata-rata jumlah item yang dipesan per order, yang penting untuk analisis volume pesanan dan pengelolaan inventaris.

B. MapReduce

Mengubah query metode agregasi ke metode mapreduce lalu membandingkan hasilnya.

1. Query 1

- Agregasi:

```
AzuraRizkyAnnisa> db.product.aggregate([
...     {$group: {_id: "$type", rate_price: {$avg: "$price"}}}
... ])
[
  { _id: 'service', rate_price: 63.333333333333336 },
  { _id: 'tv', rate_price: 50 },
  { _id: 'accessory', rate_price: 13.875 },
  { _id: 'warranty', rate_price: 38 },
  { _id: 'phone', rate_price: 260 }
]
```

- Mapreduce:

```
AzuraRizkyAnnisa> var mapFunction1 = function() {
...     emit(this.type, {sum: this.price, count: 1});
... };

AzuraRizkyAnnisa> var reduceFunction1 = function(key, values) {
...     var reducedValue = {sum: 0, count: 0};
...
...     values.forEach(function(value) {
...         reducedValue.sum += value.sum;
...         reducedValue.count += value.count;
...     });
...
...     return reducedValue;
... };

AzuraRizkyAnnisa> var finalizeFunction1 = function(key, reducedValue) {
...     return {average_price: reducedValue.sum / reducedValue.count};
... };

AzuraRizkyAnnisa> db.product.mapReduce(
...     mapFunction1,
...     reduceFunction1,
...     {
...         out: "avg_price_by_type",
...         finalize: finalizeFunction1
...     }
... );
{ result: 'avg_price_by_type', ok: 1 }
AzuraRizkyAnnisa> db.avg_price_by_type.find().pretty();
[
  { _id: 'tv', value: { average_price: 50 } },
  { _id: 'accessory', value: { average_price: 13.875 } },
  { _id: 'warranty', value: { average_price: 38 } },
  { _id: 'phone', value: { average_price: 260 } },
  { _id: 'service', value: { average_price: 63.333333333333336 } }
]
```

Pembahasan:

Perbandingan antara query di gambar 1 (menggunakan agregasi) dan gambar 2 (menggunakan MapReduce) menunjukkan dua pendekatan berbeda dalam memproses data di MongoDB. Pada gambar 1, operasi agregasi berfungsi untuk menghitung rata-rata jumlah item (AvgItemQty) per pesanan berdasarkan data pesanan yang sudah terstruktur.

Prosesnya dimulai dari $unwind untuk memisahkan setiap item dari pesanan, lalu dilakukan $group berdasarkan _id (nomor pesanan) untuk menghitung rata-rata kuantitas item di setiap order. Hasilnya menunjukkan rata-rata jumlah item yang dipesan untuk setiap order, berguna untuk analisis volume pesanan.

Sedangkan pada gambar 2, MapReduce digunakan untuk menghitung nilai rata-rata harga produk per tipe (jenis produk). Fungsi map meng-emisikan (emit) setiap produk berdasarkan tipe dan harga, dan fungsi reduce menghitung total harga dan jumlah produk per tipe. Fungsi finalize kemudian menghitung rata-rata harga per tipe. Fungsi ini cocok untuk pengolahan data yang kompleks dan kustom, seperti penghitungan statistik lanjutan.

Keunggulan agregasi adalah kecepatan dan efisiensi dalam operasi standar, cocok untuk pengelompokan dan statistik dasar. Sementara MapReduce menawarkan fleksibilitas tinggi untuk operasi kompleks dan khusus, meskipun lebih lambat dan membutuhkan lebih banyak sumber daya.

2. Query 2

- Agregasi:

```
AzuraRizkyAnnisa> db.product.aggregate([
...     {$match: {"type": "service"}},
...     {$group: {_id: "$type", num_product: {$sum: 1}}}
... ])
[ { _id: 'service', num_product: 3 } ]
```

- Mapreduce:

```
AzuraRizkyAnnisa> var mapFunction2 = function() {
...     if (this.type === "service") {
...         emit(this.type, 1);
...     }
... };

AzuraRizkyAnnisa> var reduceFunction2 = function(key, values) {
...     return Array.sum(values);
... };
...

AzuraRizkyAnnisa> db.product.mapReduce(
...     mapFunction2,
...     reduceFunction2,
...     {
...         out: "service_product_count",
...         query: {type: "service"}
...     }
... );
{ result: 'service_product_count', ok: 1 }
AzuraRizkyAnnisa> db.service_product_count.find().pretty();
[ { _id: 'service', value: 3 } ]
```

Pembahasan:

Pada gambar 1, proses agregasi berfungsi untuk menghitung jumlah produk tipe "service" dengan cara memfilter dokumen berdasarkan type tertentu, lalu mengelompokkan berdasarkan _id tipe produk dan menghitung num_product. Hasilnya menampilkan jumlah produk per tipe secara cepat dan efisien, cocok untuk operasi pengelompokan data secara standar.

Sedangkan gambar 2 menunjukkan MapReduce yang melakukan penghitungan jumlah produk bertipe "service" dengan fungsi map dan reduce. Fungsi map meng-emisikan tipe produk jika cocok, dan reduce menghitung total jumlah barang per tipe secara lebih fleksibel dan kustom, cocok untuk pengolahan data kompleks dan spesifik.

Keunggulan agregasi adalah performa lebih cepat dan sintaks yang ringkas, cocok untuk operasi statistik dan pengelompokan standar. MapReduce menawarkan fleksibilitas tinggi untuk pengolahan data yang lebih kompleks, tetapi cenderung lebih lambat dan membutuhkan effort penulisan fungsi lebih detail.

3. Query 3

- Agregasi:

```
dbmahasiswa> db.Orders.aggregate([
...     {$group: {_id: "$Country", OrderCount: {$sum: 1}}},
...     {$project: {Country: "$_id", _id: 0, OrderCount: 1}}
... ])
[ { OrderCount: 4, Country: null } ]
```

- Mapreduce:

```
dbmahasiswa> var mapFunction3 = function() {
...     emit(this.Country, 1);
... };
...

dbmahasiswa> var reduceFunction3 = function(key, values) {
...     return Array.sum(values);
... };
...
```

```
dbmahasiswa> db.Orders.mapReduce(
...     mapFunction3,
...     reduceFunction3,
...     {
...         out: "order_count_by_country",
...         finalize: function(key, reducedValue) {
...             return {Country: key, OrderCount: reducedValue};
...         }
...     }
... );
...
{ result: 'order_count_by_country', ok: 1 }
dbmahasiswa> db.order_count_by_country.find({}, {_id: 0, value: 1}).pretty()
[ { value: { Country: null, OrderCount: 4 } } ]
```

Pembahasan:

Pada operasi agregasi berfungsi untuk menghitung total order berdasarkan negara dari data order, dengan memfilter, mengelompokkan, dan menghitung jumlah order. Pendekatan ini efisien, cepat, dan cocok untuk operasi statistik dasar serta pengelompokan data yang tidak kompleks.

Sedangkan MapReduce digunakan untuk menghitung jumlah order per negara. Fungsi map mengirimkan data berdasarkan negara, dan fungsi reduce menghitung total order per negara, dengan finalize mengembalikan hasil akhir. Pendekatan ini lebih fleksibel dan mampu menjalankan pengolahan data yang lebih kompleks dan kustom, seperti penyesuaian logika penghitungan yang tidak bisa dilakukan dengan pipeline agregasi.

Keunggulan agregasi adalah performa lebih tinggi, sintaks lebih simpel, dan lebih mudah digunakan untuk operasi standar. MapReduce menawarkan keunggulan dalam menangani proses pengolahan data yang membutuhkan logika lebih kompleks dan kustom yang tidak bisa dicapai dengan agregasi, meskipun cenderung lebih lambat dan lebih kompleks dalam penulisan.

F. Kesimpulan

1. MapReduce diterapkan dengan mendefinisikan fungsi map untuk mengeluarkan pasangan kunci-nilai sesuai data yang ingin diproses, diikuti fungsi reduce untuk mengagregasi nilai berdasarkan kunci tersebut, dan optional fungsi finalize untuk hasil akhir. Kemudian, fungsi-fungsi ini digunakan dalam perintah mapReduce() di MongoDB untuk menjalankan proses pengolahan data secara terdistribusi dan kustom.
2. Menguasai prinsip MapReduce dimulai dengan memahami konsep dasar map yang memuat data ke dalam pasangan kunci-nilai, dan reduce yang memproses data tersebut menjadi hasil yang terakumulasi. Pemahaman tentang bagaimana data dipecah, diproses secara paralel, dan digabungkan kembali, serta latihan membuat berbagai fungsi sesuai kebutuhan analisis data, adalah kunci utama untuk menguasai dan mengimplementasikan MapReduce secara efektif.

DAFTAR PUSTAKA

Taqwin, P. A. T. (1970). IMPLEMENTASI METODE MAPREDUCE PADA BIG DATA BERBASIS HADOOP DISTRIBUTED FILE SYSTEM. Retrieved from https://openlibrary.telkomuniversity.ac.id/pustaka/141448/implementasi-metode-mapreduce-pada-big-data-berbasis-hadoop-distributed-file-system.html

Nagi, I. (2019). Implementasi Sederhana Framework MapReduce. Retrieved from https://medium.com/pujanggateknologi/implementasi-sederhana-framework-mapreduce-8f80f22cc54f